CS 499 Milestone Four: Databases

Artifact Enhancement Narrative

Samuel Raymond Kwibe

CS 499 Computer Science Capstone

Southern New Hampshire University

Section One: Describing the Artifact

The artifact I am using for Milestone Four is the same one I have been working with throughout this entire capstone — the MunchiesSNHU Food Waste Tracker. I originally built this application during an earlier course in the Computer Science program at Southern New Hampshire University. The app is designed to help a college campus track food waste, monitor compost bin conditions, record environmental sensor readings, and generate sustainability reports for staff and administrators to act on.

The project has a frontend built with HTML, CSS, and JavaScript and a backend running on Node.js, Express, and MongoDB. It supports multiple user roles including staff, admin, and regular users. Milestone Two focused on fixing the software structure and backend integration problems. Milestone Three added the bin priority algorithm and real data analysis features. Now in Milestone Four, I am focusing on the database layer itself — making sure data is stored correctly, queries run efficiently, duplicate models are cleaned up, and sensitive credentials are properly protected. This is the final enhancement phase, and it ties everything together by ensuring the foundation the entire application depends on is solid, secure, and well designed.

Section Two: Why I Chose This Artifact and What I Improved

I chose this artifact for the databases category because MongoDB is at the center of everything this application does. Every food waste entry, every user account, every sensor reading, and every report depends entirely on the database working correctly. If the database schema accepts bad data, the priority algorithm from Milestone Three will produce wrong scores. If the queries have no indexes, the application will slow down significantly as data grows. If duplicate models exist, data can get stored in two different places and fall out of sync without anyone noticing.

And if environment variables with database credentials are exposed in the wrong place, the entire database could be compromised. All of those are real database design problems that needed to be addressed, and fixing them in this milestone demonstrates a genuine understanding of what good database design looks like in a professional application.

Here is a detailed breakdown of everything I improved for this milestone:

- The schema validation in FoodWasteEntry.js was the first thing I verified. The Mongoose schema already had numeric range validation built in for the most critical sensor fields — pH is constrained between 0 and 14, humidity between 0 and 100, bin fullness between 0 and 100, weight cannot be negative, and gas levels cannot be negative either. This kind of schema-level validation is important because it means bad data gets rejected at the database layer before it ever gets stored. If the priority algorithm from Milestone Three receives a bin fullness value of 150 percent or a pH of negative 3, those are clearly invalid readings that would produce meaningless priority scores. Having the schema enforce those constraints automatically protects the integrity of everything the application does with that data.
- The submittedBy field was another thing I verified carefully. The FoodWasteEntry schema requires a submittedBy value — meaning every waste record needs to know which user created it. This is important for accountability and reporting purposes. I confirmed that the wasteEntryController.js already handles this correctly by automatically pulling the logged in user's ID from their JWT token using req.user._id and assigning it to submittedBy when a new entry is created. Staff never have to type their own user ID into a form — the system handles it securely behind the scenes. This is a good example of how authentication and database design work together to create a reliable and trustworthy data record.
- I added two missing database indexes to FoodWasteEntry.js — one on the published field and one on the createdAt field. The application already had some indexes in place, but these two were missing. Indexes matter because of how MongoDB searches for data.

Without an index, every query that filters by published status or sorts by creation date has to scan through every single record in the collection to find the ones that match. That works fine when the database has a few dozen records, but as the campus generates more and more waste entries over months and years, those queries will get slower and slower. Adding indexes tells MongoDB to maintain a sorted reference structure for those fields so queries can jump directly to the relevant records instead of scanning everything. I also confirmed that a compound index on both published and createdAt together exists, which optimizes the most common query pattern in the application — fetching all published entries sorted by most recent first.

- I deleted three duplicate model files that were sitting unused in the project — BinSetting.js, NotificationSetting.js, and ThemeSetting.js. These files defined separate database models for settings that are already stored directly inside the User.js model. The User schema already has fields for notification preferences and theme preferences built in. Having separate standalone models for those same things creates a dangerous situation where the same piece of user data could end up stored in two different places with two different values, and the application would have no reliable way of knowing which one was correct. Removing those duplicate files eliminated that risk and made the models folder cleaner and easier to navigate.
- I removed an exposed .env file that was sitting inside the public folder. This was probably the most urgent security fix in this entire milestone. The public folder in a web application is exactly what its name says — it is publicly accessible. Anything in that folder can potentially be seen by anyone who knows where to look. Having a .env file with database connection strings and secret keys sitting in the public folder meant those credentials could have been exposed to anyone browsing the application. I deleted that file and verified that the .gitignore file at the project root properly lists .env so that no environment files can be accidentally committed to GitHub in the future. Protecting

secrets from public exposure is a fundamental security practice that every developer needs to take seriously.

- I overhauled the reporting endpoint in foodWasteRoutes.js by adding four new MongoDB aggregation pipelines. The previous version of the reports endpoint returned some basic summaries but was missing several important breakdowns. I added total waste submitted per user using a $group stage on the submittedBy field with a $lookup to pull in the user's name, total waste per food type using a $group stage on the foodType field, a monthly waste trend over time by grouping records by year and month instead of by individual day, and the most active submission locations which groups records by location and sums their total waste. All of these aggregations run entirely at the database level using real MongoDB records — there are no calculations happening in JavaScript and no hardcoded numbers anywhere. This is the most efficient way to generate reports because MongoDB is designed to process and aggregate large collections of data much faster than application-level code can.

Section Three: Course Outcomes and Whether I Met Them

The main course outcome I was targeting in this milestone was Outcome Four — demonstrating the ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals. I am confident this outcome was fully addressed through the schema validation verification, the database indexing work, the duplicate model cleanup, the security fix for the exposed credentials, and the aggregation pipeline improvements.

Each of those improvements represents a real database design principle that matters in professional software development. Schema validation is about data integrity. Indexes are about query performance and scalability. Removing duplicate models is about maintaining a single source of truth. Protecting environment variables is about security. Aggregation pipelines are about efficient data analysis. Together they demonstrate a well-rounded understanding of what it means to design a database layer that is not just functional but professional.

Outcome Five around developing a security mindset was also directly addressed through the removal of the exposed .env file from the public folder. That was not a theoretical security concern — it was a real vulnerability in the deployed application that could have given anyone access to the database. Identifying it, understanding why it was dangerous, and removing it is exactly what a security-conscious developer does.

Outcome Two around professional communication was addressed through this narrative itself, which required explaining technical database concepts like indexes, aggregation pipelines, and schema validation clearly enough for both a technical and non-technical audience to follow.

With this milestone complete, all five CS 499 course outcomes have been addressed across the three enhancement phases. Milestones Two, Three, and Four together form a complete picture of full-stack development competency — software design and engineering, algorithms and data structures, and databases — all demonstrated through a single meaningful real-world application.

Section Four: What I Learned and What Was Hard

Going into Milestone Four I was honestly a little nervous. Databases have always been the part of full-stack development that felt the most abstract to me. Writing routes and controllers feels tangible — I can trace the flow of a request from the frontend through the backend and see exactly what is happening at each step. But database design involves thinking about things that happen invisibly in the background — how data is indexed, how schemas enforce constraints, how aggregation pipelines transform collections. Those concepts were harder to visualize and easier to get wrong without immediately noticing.

The indexing work taught me something I did not fully appreciate before this milestone. I understood conceptually that indexes make queries faster, but I had never thought carefully about which queries actually needed them in a specific application. Going through the codebase and identifying the most common query patterns — fetching published entries sorted by date, filtering by location, generating reports — and then designing indexes to support those specific patterns felt like a much more deliberate and thoughtful process than I expected. It is not just about adding indexes everywhere. Adding too many indexes has its own cost because MongoDB

has to update every index every time a record is written. The right approach is to be intentional about which queries are most critical and design indexes that serve those queries efficiently.

The most surprising moment in this milestone was discovering the exposed .env file in the public folder. I had been working with this codebase across three milestones and never noticed it was there. That discovery was a little uncomfortable because it meant a real security vulnerability had been sitting in the project the whole time without being caught. But it was also a valuable lesson about why security reviews matter and why they need to look at the whole project holistically rather than just the code that was recently changed. A file that was put in the wrong place weeks ago can create a serious problem that has nothing to do with any of the current work.

The aggregation pipelines were genuinely challenging to write correctly. MongoDB's aggregation framework is powerful but it requires thinking about data transformation in a very different way than regular queries. Each stage in the pipeline takes the output of the previous stage and transforms it further — grouping records, calculating sums, looking up related documents from other collections, projecting only the fields that are needed. Getting the stages in the right order and making sure the field names matched what the frontend was expecting took more iteration than I expected. But seeing the reports dashboard populate with real breakdowns of waste by user, by food type, and by month was genuinely satisfying.

Looking back at all three enhancement milestones together, I feel like the MunchiesSNHU project went through a real transformation. The before-enhancement snapshot shows an application with good intentions but serious structural, algorithmic, and data problems. The final milestone-two branch shows an application with a clean backend architecture, an intelligent priority algorithm, and a well-designed database layer. That progression from broken to working to well-designed to analytically useful is the kind of journey that real software development involves, and being able to document it clearly across three milestones is something I am genuinely proud of.

AI Usage Disclosure

An AI writing assistant was used to help organize and draft portions of this narrative. All content was reviewed, edited, and rewritten to reflect my own understanding, direct experience working through the artifact enhancements, and honest reflection on the challenges and lessons I learned throughout this milestone.

References

Chodorow, K. (2013). MongoDB: The definitive guide (2nd ed.). O'Reilly Media.

MongoDB. (2023). Indexes. MongoDB documentation.
https://www.mongodb.com/docs/manual/indexes/

MongoDB. (2023). Aggregation pipeline. MongoDB documentation.
https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

OWASP Foundation. (2023). Environment variables and secrets management.
https://owasp.org/www-community/vulnerabilities/Insecure_Direct_Object_Reference